Implementation Document

for

IITK ProjectSpace

Version 1.0

Prepared by

Group #: 3 **Group Name: Kernel Panic**

Abhinav Chhabra	240031	abhinavc24@iitk.ac.in
Aman Srivastava	240103	amane24@iitk.ac.in
Dhruvi Jain	240359	jdhruvi24@iitk.ac.in
Garvit Bhutani	240393	garvitb24@iitk.ac.in
Hemakshi Kumawat	240447	hemakshik24@iitk.ac.in
Laksh Dhir	240588	laksh24@iitk.ac.in
Madhusudan Kantharia	240606	madhusudan24@iitk.ac.in
Ravipati Raj Sri Vardhan	240857	rajsri24@iitk.ac.in
Shreyas Asthana	241001	shreyasa24@iitk.ac.in
Sujal Satish Motagi	241052	sujalsm24@iitk.ac.in

Course: CS253

Mentor TA: *Seetaramayya Lavu*

Date: 27th March, 2026

CONTENTS II
REVISIONS II
1 IMPLEMENTATION DETAILS 3
2 CODEBASE 5
3 COMPLETENESS 32
APPENDIX A - GROUP LOG 35

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Version 1.0	Abhinav Chhabra Aman Srivastava Dhruvi Jain Garvit Bhutani Hemakshi Kumawat Laksh Dhir Madhusudan Kantharia Ravipati Raj Sri Vardhan Shreyas Asthana Sujal Satish Motagi	First version of the Implementation Document	

1 Implementation Details

1.1 Programming Languages, Framework and Libraries

IITK ProjectSpace is an end to end project collaboration and recruitment platform built using a full modern stack architecture to provide a smooth experience for the user. The system connects users who are looking to explore projects and publish recruitment opportunities, collaborate with peers while being able to post their own projects and interact with other posts present on the platform.

1.1.1 Programming Languages

IITK ProjectSpace has been developed using a combination of TypeScript (for the frontend) and Python (for the backend). It follows a modern client-server architecture pattern. For the backend we have used,

1. Python: The benefit of using Python for the backend is its clean syntax and massive ecosystem of data science and web frameworks. It allows for rapid development and integrates seamlessly with modern API frameworks like FastAPI. It enables efficient asynchronous operations using async/await, seamless integration with powerful libraries for database management and validation, and strong scalability supported by a vast ecosystem of tools.

For the Frontend, we have used,

1. HTML/CSS: HTML is used to structure web pages, while CSS (specifically via Tailwind CSS) is employed for styling, providing a consistent, responsive, and visually appealing layout across multiple devices.

2. TypeScript: TypeScript is used alongside React/Next.js to add static typing to JavaScript. This significantly reduces runtime errors, improves code maintainability, and provides an excellent developer experience through enhanced IDE support and auto-completion.

For the Database Management System (DBMS) we have used,

1. PostgreSQL: We chose PostgreSQL, a powerful, open-source object-relational database system. Its benefits over NoSQL alternatives include robust support for ACID transactions, complex queries, and data integrity. Furthermore, PostgreSQL's advanced features, such as TSVector for efficient full-text search, were crucial for implementing the complex search functionalities (searching projects, recruitments, and users) in ProjectSpace.

1.1.2 Frameworks

Backend Frameworks

We used **FastAPI** as the backend framework because of the following reasons

- **FastAPI** is one of the fastest Python frameworks available, built on Starlette and Pydantic
- FastAPI automatically generates **interactive documentation** which improves the ease of frontend - backend integration testing.

- It natively supports **asynchronous programming** allowing it to handle multiple concurrent requests.
- Integrated Pydantic models ensure vigorous **data validation and serialization.**

Frontend Framework

For the frontend, we chose **Next.js**, which is a react framework. We chose this framework for the following reasons

- Next.js equips the webpages with optimized rendering and faster load times, ensuring a **smoother user experience**.
- Taking advantage of React, it allows the building of **reusable UI components**, making development organized.
- It uses a highly intuitive **file-based routing system**, making the integration of navigation in the webpage easier.

1.1.3 Libraries

The following libraries and tools have been used in our implementation

1. **SQLModel -** This is used as the Object-Relational Mapper (ORM) in the backend. It combines SQLAlchemy and Pydantic, providing an intuitive and type-safe way to interact with the PostgreSQL database, simplifying CRUD operations and schema management.

2. **TailwindCSS -** A utility-first CSS framework used in the frontend. It allows for rapid UI development by applying pre-existing classes directly in the markup, ensuring a consistent design system without writing custom CSS from scratch.

1.1.4 Build and Deployment Systems

1. **NPM (Node Package Manager) -** Used for the frontend to install, manage, and track dependencies, ensuring a reproducible build environment.

2. **Docker & Docker Compose -** Employed to containerize the application (frontend, backend and database). This guarantees that the software runs consistently across different environments (development, testing and production) and simplifies the deployment process by managing multi-container setups seamlessly.

1.1.5 Authentication

1. **Passlib(Bcrypt) -** This is used for hashing and salting passwords securely before storing them in the database.

2. **Jose -** This library is used for generating and validating JSON Web Tokens (JWT) for secure, stateless and session management.

3. **FastMail -** Utilized in the backend to send OTPs (One Time Passwords) to the users for secure registration and password reset

2 Codebase

GitHub Repository: IITK ProjectSpace

Code Structure

The above link takes you to the GitHub repository of IITK ProjectSpace, which contains all the source code for our web application. The project repository is mainly composed of 4 main parts:

- Frontend
- Backend
- Documentation
- Assets

Overall Structure of the repository for IITK-ProjectSpace:

IITK-ProjectSpace/
|-- backend/
|-- frontend/
|-- documentation/
|-- assets/
|-- .gitignore
|-- docker-compose.yml
|-- package.json
|-- package-lock.json #
|-- Readme.md

Frontend

After opening the frontend folder, you can see the subsequent directory structure. It has mainly the following major parts:

- **public**: Contains static assets like images, campus backgrounds (e.g., iitkbg-image.jpg), logos (Logo.png), and UI icons used throughout the IITK ProjectSpace application.

- **src**: Contains all the core source code for the application, organized into:

    - **app**: Serves as the main routing foundation using the Next.js App Router. It is organized by major features (auth, homepage, projectPage, recruitmentPage, profilePage, etc.). Each feature directory contains its own route definitions (page.tsx) and corresponding styling files (.css).
    - **components**: Contains reusable, globally shared UI elements (such as ProjectsCard, RecruitmentCard, and OverflowTooltip) to ensure a consistent design

language across the project showcasing and recruitment interfaces.

- **lib**: Contains all utility functions and API integration modules (projectApi.ts, authApi.ts, token.ts). This is where the frontend securely communicates with the backend to handle portfolio updates, searches, and recruitments.

- **data**: Stores static and seed data, such as the predefined list of technical skills/tags (seed_skills.json) used for categorizing projects and user profiles.

- **Configuration files**: Essential setup files at the root level that dictate framework behavior, code quality, and styling. This includes next.config.ts for Next.js routing/environment rules, tailwind.config.js for custom UI styling, and tsconfig.json for TypeScript type-safety.

The frontend for the project utilizes Next.js (App Router) as its core framework, written in TypeScript to ensure type safety and scalable code. The styling is primarily handled via Tailwind CSS alongside feature-specific CSS modules.

The application follows a modular, feature-based architecture with careful separation of concerns. UI components and routes are logically grouped by functionality (e.g., profilePage, projectPage, recruitmentPage) to handle the diverse needs of students, researchers, clubs, and professors.

The frontend communicates with the backend through modularized RESTful API utility functions located in the libdirectory (e.g., projectApi.ts, authApi.ts). Authentication state and tokens are managed securely, and the application features a responsive design to facilitate project showcasing, **searching, and recruitment natively without relying on external forms.**

Frontend Structure

```
|-- frontend/ // Frontend Directory
   |-- public/ // Static assets accessible directly
      |-- Logo.png // Main IITK ProjectSpace logo
      |-- file.svg // Generic file icon asset
      |-- globe.svg // Globe icon asset
      |-- iitkbg-image.jpg // IIT Kanpur campus background image
      |-- next.svg // Next.js framework asset
      |-- vercel.svg // Vercel hosting asset
      |-- window.svg // Window icon asset
   |-- src/ // Application source code
      |-- app/ // Next.js App Router root (Pages and Routing)
         |-- auth/ // Authentication & Onboarding flow
            |-- AlertPopUp.css // Styling for alert modals
            |-- AlertPopUp.tsx // Reusable alert/notification modal
            |-- loginPage.css // Styling for the login screen
            |-- otpPopUp.css // Styling for OTP verification
            |-- otpPopUp.tsx // OTP entry and validation component
```

```
        |-- page.tsx // Main authentication/login page
    |-- components/ // Global layout & interactive components
        |-- Header.css // Top navigation bar styling
        |-- Header.tsx // Top navigation bar component
        |-- NotificationDrawer.css // Notification sidebar styling
        |-- NotificationDrawer.tsx // Real-time alerts and notifications UI
        |-- Sidebar.css // Side navigation styling
        |-- Sidebar.tsx // Side navigation component
        |-- cards/ // Layout wrappers for card interfaces
        |-- commentsSection.css // Styling for project/recruitment comments
        |-- commentsSection.tsx // Interactive discussion/commenting UI
        |-- confirmPopUp.css // Confirmation dialog styling
        |-- confirmPopUp.tsx // Action confirmation modal
    |-- homepage/ // Landing and main dashboard
        |-- homePage.css // Dashboard styling
        |-- page.tsx // Main landing page rendering
    |-- postCreationForm/ // Project and Recruitment creation
        |-- page.tsx // Form for publishing new projects
        |-- postCreationForm.css // Form UI styling
    |-- profilePage/ // User portfolio & profile management
        |-- ProfilePage.css // Profile display styling
        |-- editProfilePage/ // Route for modifying user details
        |-- page.tsx // Main user portfolio display
    |-- projectPage/ // Centralized project showcase views
        |-- editProjectPage/ // Route for updating existing projects
        |-- page.tsx // Detailed view of a specific project
        |-- projectPage.css // Project detail styling
    |-- recruitmentPage/ // Dedicated project recruitment hub
        |-- editRecruitmentPage/ // Route for modifying recruitment listings
        |-- page.tsx // Detailed view of recruitment openings
        |-- recruitmentPage.css // Recruitment listing styling
    |-- searchPage/ // Global platform search functionality
        |-- SearchPage.css // Search interface styling
        |-- page.tsx // Search results and filtering view
    |-- favicon.ico // Website favicon
    |-- globals.css // Global application stylesheets and Tailwind imports
    |-- layout.tsx // Root layout wrapper for Next.js App
    |-- page.tsx // Root entry/redirect page
|-- components/ // Reusable UI elements
    |-- cards/ // Data display components
        |-- OverflowTooltip.tsx // Text truncation and tooltip helper
        |-- ProjectsCard.tsx // Summary card for project showcases
        |-- RecruitmentCard.tsx // Summary card for open recruitments
        |-- UserCard.tsx // Summary card for community member profiles
```

```
    |-- data/ // Static and seed data
        |-- seed_skills.json // Predefined list of technical skills/tags
    |-- lib/ // API integrations, Context, and Utilities
        |-- authApi.ts // API calls for login/registration/OTP
        |-- commentApi.ts // API calls for fetching/posting comments
        |-- formatTeam.ts // Utility for parsing and formatting team data
        |-- notificationApi.ts // API calls for user notifications
        |-- profileApi.ts // API calls for fetching/updating user portfolios
        |-- projectApi.ts // API calls for project CRUD operations
        |-- recruitmentApi.ts // API calls for recruitment listings and applications
        |-- searchApi.ts // API calls for global platform search
        |-- token.ts // JWT/Authentication token management utilities
        |-- utils.ts // General helper functions and formatting tools
|-- README.md // Project documentation
|-- components.json // UI component library configuration
|-- eslint.config.mjs // ESLint rules for code quality
|-- next.config.ts // Next.js build and environment configuration
|-- package-lock.json // Locked npm dependency tree
|-- package.json // Project metadata, scripts, and dependencies
|-- postcss.config.js // PostCSS configuration for Tailwind CSS processing
|-- tailwind.config.js // Tailwind CSS theme and styling configuration
|-- tsconfig.json // TypeScript compiler configuration
```

Backend

The backend folder structure reveals the following major components:

- **main.py**: The main server entry point that initializes FastAPI, configures CORS, connects routers, and sets up static file serving.
- **core**: Contains foundational configurations and utilities. Handles database connection setup (`database.py`), JWT security and hashing (`security.py`), email sending functionality (`email.py`), and API dependencies for authentication (`dependencies.py`).
- **models**: Contains SQLModel schema definitions mapping to the database entities (User, Project, Recruitment, Comment, Notification, OTP).
- **schemas**: Contains Pydantic models responsible for strict data validation, structuring API request bodies, and defining response formats.
- **crud**: Encapsulates all business logic and database operations (Create, Read, Update, Delete) separated from the routing layer for cleaner code organization.
- **routers**: API endpoints organized by functionality (authentication, user profiles, project management, recruitment workflows, messaging/comments, and global search).

The backend follows a layered architectural pattern separating routing, business logic, and data access. Authentication is implemented using JWT (JSON Web Tokens) with `bcrypt` (via passlib)

for password hashing. Advanced full-text search functionality is implemented natively using PostgreSQL's `TSVECTOR` integration.

Environment variables in the `.env` file configure sensitive information like database connections, JWT secret keys, and SMTP server credentials for email verification. The server provides comprehensive data validation, strict schema responses, and detailed HTTP error messages. File upload functionality is managed directly via FastAPI's `UploadFile` with a dedicated `uploads/` directory mapped to static file serving for project media, recruitment documents, and user profile pictures.

The application clearly separates concerns with models defining data structure, schemas managing data validation, routers handling API endpoints, core dependencies managing cross-cutting concerns like authentication, and CRUD modules implementing business logic.

Backend Structure:

```
|-- backend/
   |-- .env  // Environment variables (DATABASE_URL, SECRET_KEY, SMTP, etc.)
   |-- main.py            // Main FastAPI application entry point
   |-- requirements.txt     // Dependencies and scripts
   |-- core/
      |-- config.py       // Environment variable configuration settings
      |-- database.py      // SQLModel database engine and session setup
      |-- dependencies.py  // API dependencies
      |-- email.py        // Email functionality for OTP and password reset
      |-- security.py      // Password hashing and JWT token generation
      |-- utils.py        // Standardized enums (Designation, Department, etc.)
   |-- crud/
      |-- comments.py      // Database business logic for comment threading
      |-- notification.py  // Database business logic for user alerts
      |-- project.py       // Database business logic for project management
      |-- recruitment.py   // Database business logic for recruitment and applications
      |-- search.py        // Full-text semantic search operations
      |-- user.py         // Database business logic for user profiles
   |-- models/
      |-- comments.py      // Comments and replies table schema
      |-- notification.py  // Notifications table schema
      |-- otp.py          // OTP verification table schema
      |-- project.py       // Projects and team links table schema
      |-- recruitments.py  // Recruitments, recruiters, and applications schemas
      |-- user.py         // Users profile and authentication schema
   |-- routers/
      |-- auth.py         // Authentication, registration, and password recovery routes
```

```
    |-- comments.py      // Commenting and replying functionality routes
    |-- notifications.py // Notification fetching and status management routes
    |-- projects.py      // Project creation, editing, and team management routes
    |-- recruitments.py  // Recruitment posting and applicant tracking routes
    |-- search.py        // Global search routes for users, projects, and posts
    |-- users.py         // User profile viewing and updating routes
|-- schemas/
    |-- comments.py      // Data validation schemas for comments
    |-- notification.py  // Data validation schemas for notifications
    |-- project.py       // Data validation schemas for projects
    |-- recruitments.py  // Data validation schemas for recruitment and application
    |-- search.py        // Data validation schemas for search responses
    |-- user.py          // Data validation schemas for users and auth
|-- uploads/             // Statically served directory for uploaded files
    |-- profilePictures/ // Uploaded user avatars
    |-- Projects/        // Uploaded project media files
    |-- Recruitments/    // Uploaded recruitment media files
```

Database

The database section utilizes PostgreSQL, with tables defined via SQLModel. The logical structure consists of the following primary tables and relationships:

```
|-- database (PostgreSQL)/
    |-- users/           // Stores user profiles, credentials, and authentication data
    |-- projects/        // Stores project details, media links, and metadata
    |-- team_links/      // Junction table linking users to projects (Team Members)
    |-- recruitments/    // Stores recruitment posts and opportunities
    |-- applications/    // Links users (applicants) to specific recruitment posts
    |-- comments/        // Stores user comments on projects and recruitments
    |-- notifications/   // Stores real-time alerts and user notifications
    |-- otps/            // Stores one-time passwords for email verification/reset
```

Architecture

The IITK ProjectSpace application follows a modular layered architecture (similar to the Router–Controller–Service pattern used in modern backend frameworks). The goal is to keep different parts of the system independent, making the codebase easier to maintain, scale, and debug.

Architectural Components

- **Model (Data Layer):**

    - The models, defined using SQLModel in the `backend/models` directory, represent the database structure.
    - They map directly to PostgreSQL tables and define relationships such as users owning projects or recruitments having applications.
- **Schemas (Validation Layer):**
    - Schemas are built using Pydantic and live in the `backend/schemas` directory.
    - They validate incoming requests and shape outgoing responses, ensuring that only clean and expected data flows through the system.
- **CRUD (Business Logic Layer):**
    - The CRUD modules handle all database operations: creating, reading, updating, and deleting records.
    - By keeping this logic separate, the rest of the code doesn't need to worry about how data is actually stored or retrieved.
- **Routers(API Layer):**
    - Routers define the API endpoints using FastAPI.
    - They handle incoming requests, extract parameters, and return appropriate responses, while delegating the actual work to the underlying layers.

Key Technical Implementations of the Architecture

- **Dependency Injection**
  FastAPI's `Depends` is used extensively to manage database sessions and authentication. This keeps the code clean and avoids repetitive setup in every route.
- **Security Layer**
  Authentication is handled using JWT tokens. Protected routes verify the user's identity before allowing actions like editing projects or applying for roles.
- **Search Optimization**
  The application uses PostgreSQL's TSVECTOR for efficient full-text search across users, projects, and posts, enabling fast and relevant results.
- **File Handling**
  File uploads are managed using FastAPI's `UploadFile`, with support for serving static files such as documents and media.

Other Important Files

- **main.py (Backend)**
  This is the entry point of the backend. It initializes the FastAPI app, configures middleware (CORS), sets up static file serving, and registers all API routes.
- **core/database.py**
  Handles the database connection using SQLModel and SQLAlchemy. It creates the engine and provides a reusable session dependency for the rest of the application.
- **core/security.py & core/dependencies.py**
  These files implement authentication and authorization. Passwords are hashed using

bcrypt, JWT tokens are generated and verified, and helper functions like `get_current_user` are used to protect routes.

- **core/email.py**
  Manages sending emails via SMTP, including OTPs for user registration and password recovery.
- **lib/api (Frontend)**
  Located in `frontend/src/lib/`, these files act as a bridge between the frontend and backend. They handle API calls, attach authentication tokens, and manage error handling in a structured way.

API endpoints

The following is a summary of the core API calls in the application, organized by router:

1. Authentication

a. Request Registration OTP

Method	`POST`
URL	`/auth/request-otp`
Data	`{ iitk_email, fullname }`
Status	**Description**
`201`	OTP sent to email successfully
`400`	Email already registered / Full name is required
`500`	Server error

b. Verify OTP & Create Account

Method	`POST`
URL	`/auth/verify-otp`
Data	`{ iitk_email, otp_code, password }`
Status	**Description**
`201`	Account created successfully
`400`	No verification request found / Invalid OTP / OTP expired
`500`	Server error

c. Check OTP Validity

Method	POST
URL	/auth/check-otp
Data	{ iitk_email, otp_code, purpose }
Status	**Description**
200	OTP is valid
400	Invalid verification code / OTP expired
404	No verification request found
500	Server error

d. User Login

Method	POST
URL	/auth/login
Data	{ iitk_email, password }
Status	**Description**
200	Login successful (returns JWT access_token)
401	Invalid Email or Password
500	Server error

e. Request Password Reset

Method	POST
URL	/auth/forgot-password
Data	{ iitk_email }
Status	**Description**
200	Password reset OTP sent successfully
404	Account with this email does not exist
500	Server error

f. Reset Password

Method	POST
URL	/auth/reset-password
Data	{ iitk_email, otp_code, new_password }
Status	**Description**
200	Password has been reset successfully
400	No reset request found / Invalid OTP / OTP expired
404	User not found
500	Server error

2. User Profile Management

a. Get Current User Profile

Method	GET
URL	/users/me
Headers	Authorization: Bearer <token>
Status	**Description**
200	Success (returns current user data and skills)
401	Not authenticated
500	Server error

b. Search Users

Method	GET
URL	/users/search
Headers	Authorization: Bearer <token>
Query Params	q (min 2 chars), limit (max 20, default 10)
Status	**Description**
200	Success (returns list of matching users, excludes current user)

401	Not authenticated
500	Server error

c. Update Current User Profile

Method	PATCH
URL	/users/me
Headers	Authorization: Bearer <token>
Data	{ fullname, bio, designation, degree, department, github, linkedin, other_link1, other_link2, skills, domains }
Status	**Description**
200	Profile updated successfully (returns updated user data)
401	Not authenticated
422	Invalid data format
500	Server error

d. Upload Profile Picture

Method	POST
URL	/users/me/profile-picture
Headers	Authorization: Bearer <token>
Data	FormData with image file
Status	**Description**
200	Profile picture uploaded and URL updated
400	Filename required / File must be an image
401	Not authenticated
500	Could not save file to disk / Server error

e. Get User Profile by ID

Method	GET
URL	/users/{user_id}

Headers	`Authorization: Bearer <token>`
Status	**Description**
200	Success (returns public user profile)
401	Not authenticated
404	User not found
500	Server error

f. Get Projects of a User

Method	**GET**
URL	`/users/{user_id}/projects`
Headers	`Authorization: Bearer <token>`
Status	**Description**
200	Success (returns list of user's projects)
401	Not authenticated
404	User not found
500	Server error

g. Get Recruitments of a User

Method	**GET**
URL	`/users/{user_id}/recruitments`
Headers	`Authorization: Bearer <token>`
Status	**Description**
200	Success (returns list of user's managed recruitments)
401	Not authenticated
404	User not found
500	Server error

h. Get Own Projects

Method	GET
URL	/users/me/projects
Headers	Authorization: Bearer <token>
Status	**Description**
200	Success (returns current user's projects)
401	Not authenticated
500	Server error

i. Get Own Recruitments

Method	GET
URL	/users/me/recruitments
Headers	Authorization: Bearer <token>
Status	**Description**
200	Success (returns current user's managed recruitments)
401	Not authenticated
500	Server error

3. Project Management

a. Create Project

Method	POST
URL	/projects/
Headers	Authorization: Bearer <token>
Data	{ title, summary, description, description_format, domains, links, media_urls, team_member_ids }
Status	**Description**
201	Project created successfully (creator auto-added to team)
401	Not authenticated
422	Invalid data

500	Server error

b. Get All Projects

Method	GET
URL	/projects/
Query Params	skip (default 0), limit (default 100)
Status	**Description**
200	Success (returns list of project summaries)
500	Server error

c. Get Project Count

Method	GET
URL	/projects/count
Status	**Description**
200	Success (returns total number of projects)
500	Server error

d. Get Project Details

Method	GET
URL	/projects/{project_id}
Status	**Description**
200	Success (returns full project details including team members)
404	Project not found
500	Server error

e. Update Project

Method	PATCH

URL	`/projects/{project_id}`
Headers	`Authorization: Bearer <token>`
Data	`{ title, summary, description, description_format, domains, links, media_urls } (all optional)`
Status	**Description**
200	Project updated successfully
401	Not authenticated
403	Only team members can edit this project
404	Project not found
500	Server error

f. Delete Project

Method	**DELETE**
URL	`/projects/{project_id}`
Headers	`Authorization: Bearer <token>`
Status	**Description**
204	Project deleted successfully
401	Not authenticated
403	Only team members can delete this project
404	Project not found
500	Server error

g. Upload Project Media

Method	**POST**
URL	`/projects/{project_id}/upload`
Headers	`Authorization: Bearer <token>`
Data	`FormData with media file`
Status	**Description**
200	File uploaded and project media_urls updated

401	Not authenticated
403	Only team members can upload media
404	Project not found
500	Server error

h. Get Project Media File

Method	GET
URL	/projects/{project_id}/media/{filename}
Status	**Description**
200	Success (returns the media file)
400	Invalid filename
404	File not found

i. Invite Team Member

Method	POST
URL	/projects/{project_id}/invites/users/{user_id}
Headers	Authorization: Bearer <token>
Status	**Description**
200	Invitation sent (notification dispatched to invited user)
400	User already a team member / User already invited
401	Not authenticated
403	Only team members can invite others
404	Project or User not found
500	Server error

j. Accept Project Invitation

Method	POST
URL	/projects/{project_id}/invites/accept

Headers	`Authorization: Bearer <token>`
Status	**Description**
200	Invitation accepted, user added to team
401	Not authenticated
404	Project not found / No pending invitation found
500	Server error

k. Reject Project Invitation

Method	**POST**
URL	`/projects/{project_id}/invites/reject`
Headers	`Authorization: Bearer <token>`
Status	**Description**
200	Invitation rejected
401	Not authenticated
404	Project not found / No pending invitation found
500	Server error

l. Remove Team Member

Method	**DELETE**
URL	`/projects/{project_id}/members/{user_id}`
Headers	`Authorization: Bearer <token>`
Status	**Description**
200	Member removed from project team
400	Cannot remove the project creator
401	Not authenticated
403	Only team members can remove others
404	Project not found / User is not a team member
500	Server error

4. Recruitment Management

a. Create Recruitment Post

Method	POST
URL	/recruitments/
Headers	Authorization: Bearer <token>
Data	{ title, description, description_format, domains, prerequisites, allowed_designations, allowed_departments, links, media_urls, recruiter_ids }
Status	**Description**
201	Recruitment post created (creator auto-added as lead recruiter)
401	Not authenticated
422	Invalid data
500	Server error

b. Get All Recruitment Posts

Method	GET
URL	/recruitments/
Query Params	skip (default 0), limit (default 100)
Status	**Description**
200	Success (returns list of recruitment summaries)
500	Server error

c. Get Recruitment Count

Method	GET
URL	/recruitments/count
Status	**Description**
200	Success (returns total number of recruitment posts)
500	Server error

d. Get Recruitment Details

Method	GET
URL	/recruitments/{recruitment_id}
Status	**Description**
200	Success (returns full post including recruiters and applications)
404	Recruitment not found
500	Server error

e. Update Recruitment Post

Method	PATCH
URL	/recruitments/{recruitment_id}
Headers	Authorization: Bearer <token>
Data	{ title, description, domains, prerequisites, allowed_designations, allowed_departments, status, ... } (all optional)
Status	**Description**
200	Recruitment updated successfully
401	Not authenticated
403	Only managing recruiters can edit this post
404	Recruitment not found
500	Server error

f. Delete Recruitment Post

Method	DELETE
URL	/recruitments/{recruitment_id}
Headers	Authorization: Bearer <token>
Status	**Description**
204	Recruitment deleted successfully
401	Not authenticated

403	Only managing recruiters can delete this post
404	Recruitment not found
500	Server error

g. Upload Recruitment Media

Method	POST
URL	/recruitments/{recruitment_id}/upload
Headers	Authorization: Bearer <token>
Data	FormData with media file
Status	**Description**
200	File uploaded and recruitment media_urls updated
401	Not authenticated
403	Only managing recruiters can upload media
404	Recruitment not found
500	Server error

h. Get Recruitment Media File

Method	GET
URL	/recruitments/{recruitment_id}/media/{filename}
Status	**Description**
200	Success (returns the media file)
400	Invalid filename
404	File not found

i. Invite Recruiter

Method	POST
URL	/recruitments/{recruitment_id}/invites/users/{user_id}
Headers	Authorization: Bearer <token>

Status	Description
200	Recruiter invitation sent (notification dispatched)
400	User already a recruiter / User already invited
401	Not authenticated
403	Only managing recruiters can invite others
404	Recruitment or User not found
500	Server error

j. Accept Recruiter Invitation

Method	POST
URL	/recruitments/{recruitment_id}/invites/accept
Headers	Authorization: Bearer <token>
Status	**Description**
200	Invitation accepted, user added as recruiter
401	Not authenticated
404	Recruitment not found / No pending invitation found
500	Server error

k. Reject Recruiter Invitation

Method	POST
URL	/recruitments/{recruitment_id}/invites/reject
Headers	Authorization: Bearer <token>
Status	**Description**
200	Invitation rejected
401	Not authenticated
404	Recruitment not found / No pending invitation found
500	Server error

l. Remove Recruiter

Method	DELETE
URL	/recruitments/{recruitment_id}/recruiters/{user_id}
Headers	Authorization: Bearer <token>
Status	**Description**
200	Recruiter removed from recruitment team
400	Cannot remove the recruitment creator
401	Not authenticated
403	Only managing recruiters can remove others
404	Recruitment not found / User is not a recruiter
500	Server error

m. Apply to Recruitment

Method	POST
URL	/recruitments/{recruitment_id}/applications
Headers	Authorization: Bearer <token>
Data	{ recruitment_id, message (optional) }
Status	**Description**
201	Application submitted (notification sent to creator)
400	Recruitment is closed / URL ID does not match payload ID
401	Not authenticated
404	Recruitment not found
500	Server error

n. Update Application Status (Accept / Reject)

Method	PATCH
URL	/recruitments/{recruitment_id}/applications/{application_id}
Headers	Authorization: Bearer <token>
Data	{ status } (Accepted / Rejected / Pending)

Status	Description
200	Application status updated (notification sent to applicant)
400	Application does not belong to this recruitment
401	Not authenticated
403	Only managing recruiters can update applications
404	Recruitment or Application not found
500	Server error

5. Comments & Discussions

a. Post a Comment

Method	POST
URL	/comments/
Headers	Authorization: Bearer <token>
Data	{ content, project_id (optional), recruitment_id (optional), parent_id (optional for replies) }
Status	**Description**
201	Comment posted (notification sent to post creator or parent comment author)
400	Parent comment does not belong to the same post
401	Not authenticated
404	Project / Recruitment / Parent comment not found
500	Server error

b. Get Comments for a Project

Method	GET
URL	/comments/project/{project_id}
Query Params	skip (default 0), limit (default 20, max 100)
Status	**Description**
200	Success (returns paginated top-level comments with reply counts)

404	Project not found
500	Server error

c. Get Comments for a Recruitment

Method	GET
URL	/comments/recruitment/{recruitment_id}
Query Params	skip (default 0), limit (default 20, max 100)
Status	**Description**
200	Success (returns paginated top-level comments with reply counts)
404	Recruitment not found
500	Server error

d. Get Replies to a Comment

Method	GET
URL	/comments/{comment_id}/replies
Query Params	skip (default 0), limit (default 5, max 20)
Status	**Description**
200	Success (returns paginated replies with total count)
404	Comment not found
500	Server error

e. Get Single Comment

Method	GET
URL	/comments/{comment_id}
Status	**Description**
200	Success (returns comment details)
404	Comment not found
500	Server error

f. Delete Comment

Method	DELETE
URL	/comments/{comment_id}
Headers	Authorization: Bearer <token>
Status	**Description**
204	Comment deleted (replies cascade-deleted automatically)
401	Not authenticated
403	Only comment author or post creator can delete
404	Comment not found
500	Server error

6. Notifications

a. Get User Notifications

Method	GET
URL	/notifications/
Headers	Authorization: Bearer <token>
Query Params	limit (default 20), offset (default 0)
Status	**Description**
200	Success (returns paginated notifications with total and unread_count)
401	Not authenticated
500	Server error

b. Mark Notification as Read

Method	PATCH
URL	/notifications/{notification_id}/read
Headers	Authorization: Bearer <token>

Status	Description
200	Notification marked as read
401	Not authenticated
404	Notification not found
500	Server error

c. Mark All Notifications as Read

Method	PATCH
URL	/notifications/read-all
Headers	Authorization: Bearer <token>
Status	**Description**
200	All unread notifications marked as read
401	Not authenticated
500	Server error

7. Global Search

a. Search Users

Method	GET
URL	/search/users
Headers	Authorization: Bearer <token>
Query Params	q, designation[], degree[], department[], skill[], domain[], limit (default 20), offset (default 0)
Status	**Description**
200	Success (returns paginated user results via PostgreSQL TSVECTOR search)
401	Not authenticated
500	Server error

b. Search Projects

Method	`GET`
URL	`/search/projects`
Headers	`Authorization: Bearer <token>`
Query Params	`q, domain[], skill[], limit (default 20), offset (default 0)`
Status	**Description**
200	Success (returns paginated project results)
401	Not authenticated
500	Server error

c. Search Recruitments

Method	`GET`
URL	`/search/recruitments`
Headers	`Authorization: Bearer <token>`
Query Params	`q, domain[], designation[], department[], skill[], prerequisite[], status (default Open), limit, offset`
Status	**Description**
200	Success (returns paginated recruitment results)
401	Not authenticated
500	Server error

3 Completeness

Implemented Features

The project includes a fully functional platform, IITK ProjectSpace, designed to enable students to post and explore projects as well as recruit collaborators. The platform supports post creation for projects and recruitments, along with user interaction features to facilitate collaboration. Below is the list of all implemented features:

1. **User Authentication and Account Security**
   - User authentication is implemented through `routers/auth.py`, which manages registration, login, OTP verification, and password recovery workflows.
   - Password hashing and JWT token generation are handled in `core/security.py` to ensure secure account access.
   - OTP-related email communication is managed in `core/email.py`, supporting registration verification and account recovery.
   - Authentication dependencies in `core/dependencies.py` ensure that protected routes are accessible only to verified users.
   - User credentials and authentication-related information are modeled in `models/user.py`, while request and response validation is defined in `schemas/user.py`.

2. **Profile Management**
   - User profile creation, viewing, and updates are managed through `routers/users.py`.
   - The corresponding database operations are implemented in `crud/user.py`.
   - Profile-related data structures are defined in `models/user.py`, and validation rules are handled in `schemas/user.py`.
   - Uploaded profile pictures are stored in `uploads/profilePictures/`, allowing users to personalize their profiles.
   - The frontend profile interface is implemented in `src/app/profilePage/page.tsx`.

3. **Project Posting and Management**
   - Project creation, editing, deletion, and team management are implemented in `routers/projects.py`.
   - The business logic for project operations is handled in `crud/project.py`.
   - Project data is structured using `models/project.py`, while input and output validation is defined in `schemas/project.py`.
   - Project-related media files are stored in `uploads/Projects/`, supporting rich project listings on the platform.

- The frontend project detail and management pages are implemented in `src/app/projectPage/page.tsx` and `src/app/postCreationForm/page.tsx`.

4. **Recruitment Posting and Application**
   - Recruitment-related functionality is implemented in `routers/recruitments.py`.
   - This file manages recruitment post creation, updates, and applicant tracking.
   - The corresponding database operations are handled in `crud/recruitment.py`.
   - Recruitment data models are defined in `models/recruitments.py`, and validation is provided in `schemas/recruitments.py`.
   - Recruitment attachments and related uploaded content are stored in `uploads/Recruitments/`.
   - The frontend recruitment detail page is implemented in `src/app/recruitmentPage/page.tsx`.

5. **Comments and Discussion System**
   - The commenting and reply system is implemented in `routers/comments.py`.
   - Comment threading and related business logic are handled in `crud/comments.py`.
   - Comment data structures are defined in `models/comments.py`, while validation rules are defined in `schemas/comments.py`.
   - This feature allows users to interact with project and recruitment posts through structured discussions.
   - The frontend comment interface is implemented in `src/app/components/commentsSection.tsx`.

6. **Notifications System**
   - User notifications are managed through `routers/notifications.py`.
   - Notification creation, retrieval, and update operations are handled in `crud/notification.py`.
   - Notification data is structured in `models/notification.py`, with validation defined in `schemas/notification.py`.
   - This feature helps users stay informed about platform activity and relevant interactions.
   - The frontend notification panel is implemented in `src/app/components/NotificationDrawer.tsx`.

7. **Global Search Functionality**
   - Platform-wide search is implemented in `routers/search.py`.
   - The search logic is handled in `crud/search.py`, which supports efficient retrieval of users, projects, and recruitment posts.

- Search input and output validation are defined in `schemas/search.py`.
- Search-related utility behavior is supported through the backend structure to improve discoverability across the platform.
- The frontend search interface is implemented in `src/app/searchPage/page.tsx` and connected through `src/lib/searchApi.ts`.

8. Core Application Setup and Configuration

- The backend application is initialized in `main.py`, which serves as the main FastAPI entry point.
- Database connectivity and session handling are configured in `core/database.py`.
- Environment-based configuration is managed through `core/config.py`.
- Standard enums and shared utility definitions are stored in `core/utils.py`.
- Static file serving for uploaded content is also configured through the main application setup.

9. Frontend Authentication Flow

- The authentication and onboarding flow is implemented in `src/app/auth/page.tsx`.
- OTP entry and verification are handled through `src/app/auth/otpPopUp.tsx`.
- Alerts and feedback messages are shown using `src/app/auth/AlertPopUp.tsx`.
- Styling for the authentication pages is managed through `src/app/auth/loginPage.css` and `src/app/auth/otpPopUp.css`.
- This flow ensures a smooth and secure user onboarding experience.

10. Frontend Navigation and Layout

- The top navigation bar is implemented in `src/app/components/Header.tsx`.
- The side navigation panel is implemented in `src/app/components/Sidebar.tsx`.
- Global layout handling is managed through `src/app/layout.tsx`, while the root entry flow is controlled by `src/app/page.tsx`.
- These components provide a consistent and structured navigation experience across the application.

11. Reusable UI Components and Cards

- Reusable profile, project, and recruitment cards are implemented in `src/components/cards/UserCard.tsx`, `ProjectsCard.tsx`, and `RecruitmentCard.tsx`.

- Long text handling and compact display are supported through `src/components/cards/OverflowTooltip.tsx`.
- These reusable components improve consistency and reduce duplication across the UI.

12. **Styling, Assets, and Frontend Configuration**
    - Global styling is maintained through `src/app/globals.css`, page-specific CSS files, Tailwind CSS, and `tailwind.config.js`.
    - The UI component system is configured through `components.json`, supporting modern reusable design elements.
    - Static assets such as `Logo.png` and `iitkbg-image.jpg` are stored in `public/` to maintain a polished visual identity.
    - The frontend is built using Next.js, React, and TypeScript for a modular and maintainable user experience.

Future Development Plans

1. **Attaching CV/ Resume to Recruitment Application**
   We plan to add the feature of attaching your CV / Resume to the application of Recruitment Posts, so that it would make the process of recruitment smoother for the recruiter to judge the capabilities of the applicant based on their CV / Resume

2. **Adding color coded designations**
   We plan to add the feature of color coded designations. This would help the user in identifying a person's designation just by looking at the color coded emphasis wherever their name is mentioned. This feature would give more prestigious and prominent colors to higher designations like Professors and Higher Academic Grade Professors.

Appendix A - Group Log

Date	Timings	Discussion & Work done
12th Feb, 2026	21:00 - 23:00	**Project Kickoff:** Discussed the scope of IITK ProjectSpace. Finalized the architecture and tech stack (Next.js, TypeScript, TailwindCSS, FastAPI, PostgreSQL). Divided initial tasks among team members.
19th Feb, 2026	22:00 - 01:00	**Project Setup & Database Schema:** Initialized the GitHub repository. Set up the Next.js App Router for the frontend and the FastAPI environment for the backend. Defined the database schema and models using SQLModel.
2nd Mar, 2026	20:00 - 23:30	**Backend Auth & Core APIs:** Implemented user authentication with JWT, configured password hashing (passlib/bcrypt), and integrated FastMail for OTP verification. Created core database logic for users and profiles.
9th Mar, 2026	21:00 - 01:30	**Frontend UI & Feature Endpoints:** Built reusable UI components (ProjectsCard, UserCard) and global layouts (Sidebar, Header) using TailwindCSS. Developed backend routers for Projects and Recruitments.
13th Mar, 2026	20:00 - 02:00	**Integration & Global Search:** Connected frontend API utility functions (`lib/projectApi.ts`, etc.) to the backend. Implemented the complex PostgreSQL TSVECTOR global search functionality and handled file uploads for media.
18th Mar, 2026	22:00 - 01:30	**Advanced Features & Debugging:** Integrated the real-time notification drawer and the nested comments/discussion system. Debugged file serving, UI routing issues, and recruitment application state changes.
23rd Mar, 2026	18:00 - 21:00	**Final Polish and Documentation:** Conducted end-to-end testing of the recruitment workflow and project creation forms. Fixed minor UI inconsistencies and collaborated to draft the final Implementation Document.
26th Mar, 2026	18:00 - 19:00	**Discussion with TA:** Met with our TA to demonstrate current progress. Discussed challenges being faced and received valuable feedback.